

Emo AI Orchestrator

Multi-Agent Intelligence Orchestration System

Project Reference Document

Version 1.0

Prepared by: Eng. Emam Abdullaziz

Table of Contents

1. Project Overview
2. System Architecture
3. Technology Stack
4. Project Structure & Files
5. Core Components
 - 5.1 main.py - FastAPI Server
 - 5.2 brain.py - LLM Interface
 - 5.3 agent.py - Agent System
 - 5.4 tools.py - Tool Registry (68 Tools)
 - 5.5 memory.py - Memory System
 - 5.6 i18n.py - Internationalization
 - 5.7 templates/index.html - Web UI
6. Complete Tools List
7. API Endpoints
8. Installation & Setup
9. Credits

1. Project Overview

Emo AI Orchestrator is a multi-agent AI orchestration system designed to distribute complex tasks among specialized AI agents. The system serves as a smart intermediary that receives user requests, analyzes them using an LLM planner, breaks them into subtasks, assigns each subtask to the most appropriate agent, and aggregates the results into a coherent final response.

The system features a modern web interface built with FastAPI and TailwindCSS, supporting both Arabic and English languages, real-time execution logging via Server-Sent Events (SSE), and a comprehensive library of 68 tools organized into 11 categories.

Key Features

- Multi-agent architecture with 4 specialized agents (Planner, Coder, Writer, Researcher)
- Hybrid task routing: LLM classifies tasks, routes to specialized agents
- Dynamic execution: parallel (threading) and sequential task execution
- 68 built-in tools across 11 categories
- Dual LLM provider support: OpenRouter and Groq
- Arabic/English bilingual interface with RTL support
- Dark/Light theme support
- Real-time execution log streaming via SSE
- Persistent memory with ChromaDB vector store
- Modern responsive web UI built with TailwindCSS

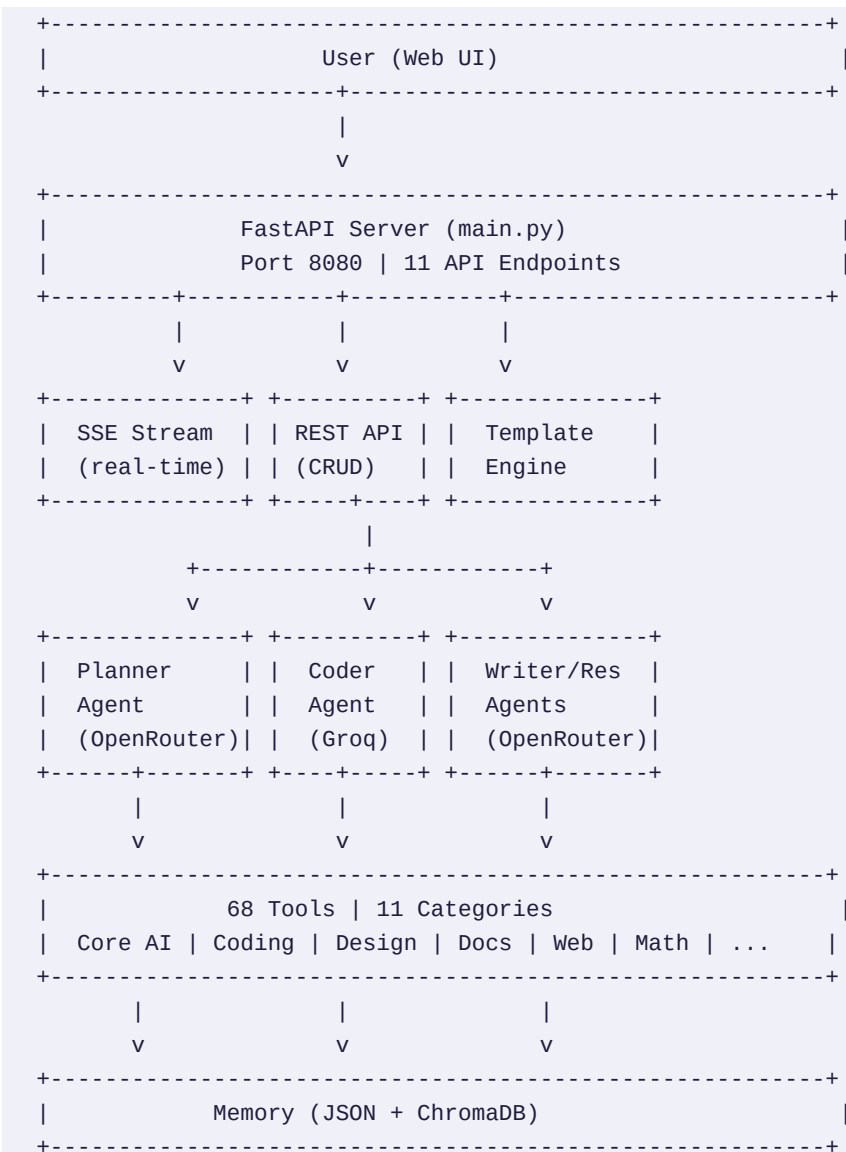
2. System Architecture

The system follows a hub-and-spoke architecture where the Orchestrator (main.py) acts as the central coordinator. User requests flow through the following pipeline:

Request Flow

User Input -> Planner Agent (classify & break down) -> Agent Assignment -> Parallel/Sequential Execution -> Result Aggregation -> Final Response

Architecture Diagram



3. Technology Stack

Backend Framework:

- FastAPI (Python 3.14)
- Uvicorn 0.47.0
- Custom (regex-based)
- OpenAI Python SDK v2.37.0
- OpenRouter (GPT-4o-mini) / Groq (Llama 3.3-70B)
- ChromaDB (PersistentClient)
- TailwindCSS (CDN)
- Font Awesome 6.5.1
- Inter / Cairo (Google Fonts)
- fpdf2
- Server-Sent Events (SSE)

Web S
Temp
LLM I
LLM I
Vecto
Front
Icons
Fonts
PDF C
Real-

4. Project Structure & Files

```

ai-agent-controller/
|-- main.py           FastAPI server + APIs + template engine
|-- brain.py          LLM interface (OpenRouter/Groq)
|-- agent.py          Agent classes + provider mapping
|-- tools.py          68 tools in 11 categories
|-- memory.py         JSON + ChromaDB memory store
|-- i18n.py           Arabic/English translations
|-- generate_pdf.py   PDF document generator
|-- .emo_settings.json Persisted settings (auto-created)
|-- templates/
|   +-- index.html    Web UI (TailwindCSS + JS)
|-- static/           Static assets directory
|-- .memory/          Memory store (auto-created)
+-- venv/             Python virtual environment

```

File Descriptions

main.py

FastAPI application with custom regex-based template engine. Serves the web UI (GET /), manages settings (GET/POST /api/settings), processes chat messages asynchronously (POST /api/chat), streams execution logs via SSE (GET /api/stream/{task_id}), lists tools and history.

brain.py

LLM abstraction layer. Supports OpenRouter and Groq providers through OpenAI-compatible API. Uses lazy initialization - API keys are only checked at first use, allowing the server to start without configured keys. Reads OPENROUTER_API_KEY and GROQ_API_KEY from environment or .env file.

agent.py

Defines 4 agent types: Planner (classifies tasks, outputs JSON plan), Coder (writes/debugs code via Groq), Writer (docs/text via OpenRouter), Researcher (web search via OpenRouter). Each agent has a custom system prompt, allowed tools list, and provider assignment.

tools.py

Comprehensive tool registry with 68 tools across 11 categories. Each tool extends the Tool base class with name, description, category, icon, and run() method. Tools range from simple (Calculator, StatsTool) to complex (WebSearch with real DuckDuckGo scraping, ScreenCapture via macOS screenshot).

memory.py

Two-tier memory system: Memory class provides JSON file-based persistence for simple key-value storage with search/recent methods. VectorMemory class (optional, requires chromadb) provides semantic search via ChromaDB

persistent client.

i18n.py

Internationalization module with complete English and Arabic translations for all UI strings. Exposes T dictionary keyed by language code and a helper function t(key, lang).

5.1 main.py - FastAPI Server

The entry point and HTTP server. Built with FastAPI, it exposes 11 routes including the main page, RESTful settings management, chat processing with background threading, and Server-Sent Events streaming for real-time execution logs.

Key Classes & Functions

- AppState: Singleton holding tools registry, agents, memory, settings, active tasks, and chat history
- render_template(): Custom regex-based template engine replacing {{ var }} placeholders
- _process_task(): Background thread that runs the full orchestration pipeline
- _parse_plan(): Extracts JSON task plan from LLM planner output with fallback

Chat Processing Flow

1. POST /api/chat creates a task ID and spawns a background thread
2. The thread calls Planner agent to analyze the request
3. Parse the JSON plan into task list
4. Execute each task through the appropriate agent (with threading for parallel groups)
5. Aggregate results using a dedicated aggregator agent
6. Store result and chat history
7. Client receives real-time updates via GET /api/stream/{task_id} (SSE)

5.2 brain.py - LLM Interface

Provides a unified interface to multiple LLM providers. Currently supports OpenRouter (default, model: gpt-4o-mini) and Groq (model: llama-3.3-70b-versatile). Uses lazy initialization to defer API key validation until the first ask() call.

Provider Configuration

```
PROVIDERS = {
    "openrouter": {
        "base_url": "https://openrouter.ai/api/v1",
        "api_key_env": "OPENROUTER_API_KEY",
        "default_model": "gpt-4o-mini",
    },
    "groq": {
        "base_url": "https://api.groq.com/openai/v1",
        "api_key_env": "GROQ_API_KEY",
        "default_model": "llama-3.3-70b-versatile",
    },
}
```

The Brain class exposes an ask() method accepting system prompt, user prompt, temperature (default 0.7), and max_tokens. Errors are caught and returned as strings to prevent crashes.

5.3 agent.py - Agent System

Defines the Agent class and factory function. Each agent has a type (planner, coder, writer, researcher), an assigned LLM provider, a system prompt, and a list of allowed tools. Agents can run tasks with or without tool access.

Agent Configuration

```
AGENT_PROVIDERS = {
    "planner": "openrouter",    # Classification & planning
    "coder": "groq",            # Code generation (fast)
    "writer": "openrouter",     # Documentation & text
    "researcher": "openrouter", # Web research
}

AGENT_TOOLS = {
    # Each agent type has access to specific tools
    "coder": ["code_writer", "run_python", "shell_execute",
              "file_reader", "file_writer", ...],
    "writer": ["doc_writer", "report_generator", "email_writer", ...],
    "researcher": ["web_search", "web_scraper", "fact_checker", ...],
}
```

Tool Execution

When an agent runs with tools, the system prompt includes available tool descriptions and usage format. The agent can invoke tools using TOOL:/ENDTOOL markers in its response. The _execute_tool_calls() method parses these markers, calls the appropriate tool, and injects results back into the response.

5.4 tools.py - Tool Registry (68 Tools)

The tool system uses a registry pattern. Each tool extends the Tool base class and registers itself with a name, description, category, and icon. The ToolRegistry provides lookup by name and organization by category.

Tool Base Class

```
class Tool:
    name = ""           # Unique identifier
    description = ""     # Human-readable description
    category = "General" # Category grouping
    icon = ""           # Emoji icon for UI
    parameters = {}      # Parameter schema

    def run(self, **kwargs) -> str:
        raise NotImplementedError

    def to_dict(self):
        return { ... } # Serialize for API
```

The create_registry() factory function instantiates and registers all 68 tools. The registry is accessible via API at GET /api/tools which returns all tools organized by category with icons and descriptions.

6. Complete Tools List

Core AI (6 tools)

- llm_chat
- planner
- router
- memory_save
- memory_search
- context_builder

Coding (7 tools)

- code_writer
- code_explainer
- code_debugger
- code_refactor
- code_review
- test_generator
- run_python

Design (6 tools)

- ui_designer
- ux_suggester
- color_palette_generator
- layout_builder
- image_prompt_builder
- branding_tool

Documents (7 tools)

- doc_writer
- doc_summarizer
- doc_formatter
- report_generator
- email_writer
- cv_builder
- ppt_outline_builder

Web & Search (4 tools)

- web_search
- web_scraper
- url_reader
- fact_checker

Math & Data (5 tools)

- calculator
- data_analyzer
- csv_reader
- json_parser
- stats_tool

Management (5 tools)

- task_manager
- goal_tracker
- decision_maker
- priority_ranker
- self_reflection

File System (10 tools)

- file_reader
- file_writer
- file_editor
- file_organizer
- folder_creator
- file_search
- file_delete
- file_move
- file_copy
- folder_scan

Screen & Vision (5 tools)

- screen_capture
- screen_analyze
- ui_element_detector
- screen_ocr
- screen_watch_mode

Browser (9 tools)

- browser_open
- browser_navigate
- browser_click
- browser_type
- browser_scroll
- browser_extract_text
- browser_take_screenshot
- browser_login_helper
- browser_automation_script

System (4 tools)

- system_info
- process_list
- shell_execute
- background_task

7. API Endpoints

GET /

Response:

HTML Response

Main web interface

GET /api/settings

Response:

JSON

Get current settings (lang, theme, API keys, etc.)

POST /api/settings

Response:

JSON

Update a setting by key/value

GET /api/tools

Response:

JSON

List all 68 tools organized by category

GET /api/history

Response:

JSON

Get chat history (last 50 messages)

POST /api/chat

Response:

JSON

Send a message, returns task_id for streaming

GET /api/stream/{task_id}

Response:

SSE

Server-Sent Events stream for execution log

8. Installation & Setup

Prerequisites

- Python 3.14+
- OpenRouter API key (<https://openrouter.ai>)
- Groq API key (<https://groq.com>)

Setup Steps

1. Clone or navigate to the project directory:

```
cd ai-agent-controller
```

2. Create and activate virtual environment (already done):

```
python3 -m venv venv  
source venv/bin/activate
```

3. Install dependencies:

```
pip install fastapi uvicorn openai chromadb jinja2 python-multipart python-dotenv fpdf2
```

4. Set API keys (either method):

```
# Method 1: Export environment variables  
export OPENROUTER_API_KEY="sk-or-v1-..."  
export GROQ_API_KEY="gsk_..."  
export LLM_PROVIDER="openrouter" # or "groq"  
  
# Method 2: Create .env file  
# OPENROUTER_API_KEY=sk-or-v1-...  
# GROQ_API_KEY=gsk_...
```

5. Run the server:

```
python3 main.py
```

6. Open your browser to:

```
http://localhost:8080
```

Alternative: Using the virtual environment directly

```
./venv/bin/python3 main.py
```

Setting API Keys via the Web UI

Once the server is running, you can set API keys directly from the Settings panel in the left sidebar. Enter your keys and click Save. The server will automatically recreate agents with the new credentials.

9. Credits

Project Development

Eng. Emam Abdullaziz

زي زعل ادب ع مام ا س دن هم ا

System Architecture & Design: Eng. Emam Abdullaziz

Full Stack Development: Eng. Emam Abdullaziz

AI Integration & Agent Design: Eng. Emam Abdullaziz

UI/UX Design: Eng. Emam Abdullaziz

Thank you for using Emo AI Orchestrator

2026